ğrencibilgiLab 04 – Hafta 4Qt Framework'üne Giriş

Lab Hakkında

Bu laboratuvar çalışmasında Qt6 framework'ünün temel yapı taşlarını uygulamalı olarak öğreneceksiniz: pencere oluşturma, QObject hiyerarşisi, sinyal/slot mekanizması ve widget'larla form tasarımı. Her çalışma bölümünde önce örnek kodu inceleyip çalıştırın, ardından görevleri tamamlayın.

Süre: 2 saat **Gereksinimler:** Qt6, CMake 3.16+, C++20 destekli derleyici

Qt Projesi Derleme

Tüm çalışmalarda aşağıdaki CMake yapısını kullanın:

```
cmake_minimum_required(VERSION 3.16)
project(LabCalisma LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
find_package(Qt6 REQUIRED COMPONENTS Widgets)
qt_standard_project_setup()
qt_add_executable(app main.cpp)
target_link_libraries(app PRIVATE Qt6::Widgets)
```

Derleme komutları:

```
mkdir build && cd build
cmake ..
cmake --build .
```

1 Çalışma 1: İlk Qt Penceresi (25 dk)

Ön Bilgi

Her Qt GUI uygulaması bir `QApplication` nesnesi ile başlar. Bu nesne olay döngüsünü (event loop) yönetir. Ekrana görünen her öğe bir `QWidget`'tır – pencereler, butonlar, metin kutuları hepsi birer widget'tır. Bir widget'ın parent'ı yoksa bağımsız bir pencere olarak görünür.

Aşağıdaki başlangıç kodunu `main.cpp` dosyasına yazın ve derleyin:

```
#include <QApplication>
#include <QWidget>
#include <QLabel>

int main(int argc, char* argv[]) {
    QApplication app(argc, argv);

    // Ana pencere oluştur
    QWidget pencere;
    pencere.setWindowTitle("OBYS - Ilk Pencere");
    pencere.resize(400, 300);

    // Pencere icine bir etiket ekle
    QLabel etiket("Merhaba Qt!", &pencere);
    etiket.setGeometry(150, 130, 200, 30);

    pencere.show();
    return app.exec();
}
```

`QApplication::exec()` Olay Döngüsü

`app.exec()` çağrısı programı bir olay döngüsüne sokar. Pencere kapatılana kadar program bu döngüde kalır ve kullanıcı etkileşimlerini (fare tıklaması, klavye girişi, pencere yeniden boyutlandırma vb.) işler. Bu satır olmadan pencere açılıp hemen kapanır.

Görev 1.1 – Pencere Özelleştirme (10 dk)

Başlangıç kodunu aşağıdaki şekilde genişletin:

a) Pencere boyutunu `600x400` piksel olarak ayarlayın.

b) Pencere başlığını `"OBYS - Öğrenci Bilgi Yonetim Sistemi"` olarak değiştirin.

c) `setMinimumSize(400, 300)` ile minimum boyut belirleyin.

d) `setMaximumSize(800, 600)` ile maksimum boyut belirleyin.

Görev 1.2 – Birden Fazla Etiket (15 dk)

Pencereye birden fazla `QLabel` ekleyin:

a) Pencerenin üst kısmına `"OBYS Giris Ekrani"` başlık etiketi ekleyin (`setGeometry` ile konumlandırın).

b) Başlık etiketinin yazı tipini büyütün:

```
baslik.setStyleSheet("font-size: 18px; font-weight: bold;");
```

c) Altına `"Öğrenci No:"` ve `"Sifre:"` etiketleri ekleyin.

d) Tüm etiketlerin parent'ını `&pencere` olarak ayarlayın.

Dikkat: Parent-Child Bellek Yönetimi

Qt'da bir widget'ın parent'ını belirlerseniz, parent yok edildiğinde child'lar da otomatik olarak yok edilir. Bu nedenle `new` ile oluşturduğunuz widget'ları `delete` etmenize gerek yoktur – parent bunu sizin yerinize yapar.

2 Çalışma 2: Parent-Child ve QObject (25 dk)

Ön Bilgi

`QObject` tüm Qt nesnelerinin temel sınıfıdır. Her `QObject` bir parent-child ağacı oluşturur. `children()` metodu ile bir nesnenin alt nesnelerine, `findChild<T>()` ile belirli türdeki alt nesneye ulaşabilirsiniz. Bu hiyerarşi bellek yönetimi, olay dağıtımı ve nesne keşfi için kullanılır.

Aşağıdaki kodu `main.cpp` dosyasına yazın:

```cpp
#include <QApplication>
#include <QWidget>
#include <QPushButton>
#include <QLabel>
#include <QDebug>

int main(int argc, char* argv[]) {
    QApplication app(argc, argv);

    // Ana pencere (parent)
    auto* pencere = new QWidget;
    pencere->setWindowTitle("QObject Hiyerarsisi");
    pencere->resize(400, 300);

    // Child widget'lar
    auto* baslik = new QLabel("OBYS Ana Menu", pencere);
    baslik->setObjectName("baslikLabel");
    baslik->setGeometry(120, 20, 200, 30);

    auto* buton1 = new QPushButton("Öğrenci Listesi", pencere);
    buton1->setObjectName("listeButon");
    buton1->setGeometry(120, 70, 160, 35);

    auto* buton2 = new QPushButton("Not Girisi", pencere);
    buton2->setObjectName("notButon");
    buton2->setGeometry(120, 120, 160, 35);

    // Hiyerarsiyi incele
    qDebug() << "Cocuk sayisi:" << pencere->children().size();
    for (auto* child : pencere->children()) {
        qDebug() << " -" << child->objectName()
                 << "(" << child->metaObject()->className() << ")";
    }

    // findChild ile belirli bir nesneyi bul
    auto* bulunan = pencere->findChild<QPushButton*>("notButon");
    if (bulunan) {
        qDebug() << "Bulunan buton:" << bulunan->text();
    }

    pencere->show();
    return app.exec();
}
```

`qDebug()` ve `objectName`

`qDebug()` Qt'un debug çıktı aracıdır. Konsola mesaj yazdırır ve QString, QObject gibi Qt türlerini doğrudan destekler. `setObjectName()` her nesneye benzersiz bir isim verir ve `findChild()` ile arama yapılabilir hale getirir.

Görev 2.1 – QObject Hiyerarşisi Keşfi (10 dk)

Başlangıç kodunu genişleterek:

a) Pencereye 2 adet daha `QPushButton` ekleyin: `"Devamsizlik"` ve `"Cikis"` butonları.

b) Her butona `setObjectName()` ile anlamlı bir isim verin.

c) `children()` ile tüm child nesneleri listeleyin ve her birinin sınıf adını (`metaObject()->className()`) yazdırın.

d) Toplam child sayısını `qDebug()` ile yazdırın.

Görev 2.2 – findChild ve findChildren (15 dk)

Nesne arama fonksiyonlarını kullanarak:

a) `findChild<QLabel*>()` ile başlık etiketini bulun ve metnini yazdırın.

b) `findChildren<QPushButton*>()` ile tüm butonları bulun:

```
auto butonlar = pencere->findChildren<QPushButton*>();
qDebug() << "Buton sayisi:" << butonlar.size();
for (auto* b : butonlar) {
    qDebug() << " Buton:" << b->text();
}
```

c) İsmi `"cikisButon"` olan butonu `findChild` ile bulun ve metnini `"Sistemi Kapat"` olarak değiştirin.

3 Çalışma 3: Sinyal/Slot Bağlantıları (30 dk)

Ön Bilgi

Qt'un en güçlü özelliği sinyal/slot mekanizmasıdır. Bir nesne durum değişikliğini **sinyal** göndererek bildirir; ilgili nesneler bu sinyali **slot** fonksiyonlarıyla yakalar. `connect()` fonksiyonu sinyalleri slotlara bağlar. C++11 lambda ifadeleri de doğrudan slot olarak kullanılabilir.
Yeni syntax (derleme zamanı kontrolü): `connect(kaynak, &Sinif::sinyal, hedef, &Sinif::slot);`

Aşağıdaki kodu `main.cpp` dosyasına yazın:

```
#include <QApplication>
#include <QWidget>
#include <QPushButton>
#include <QLabel>
#include <QDebug>

int main(int argc, char* argv[]) {
    QApplication app(argc, argv);

    QWidget pencere;
    pencere.setWindowTitle("Sinyal/Slot Ornegi");
    pencere.resize(400, 300);

    auto* etiket = new QLabel("Butona tiklayin", &pencere);
    etiket->setGeometry(120, 30, 200, 30);

    auto* sayacButon = new QPushButton("Tikla: 0", &pencere);
    sayacButon->setGeometry(120, 80, 160, 40);

    auto* kapatButon = new QPushButton("Kapat", &pencere);
    kapatButon->setGeometry(120, 140, 160, 40);

    // Lambda ile sinyal/slot bağlantısi
    int sayac = 0;
    QObject::connect(sayacButon, &QPushButton::clicked,
        [&sayac, sayacButon, etiket]() {
            sayac++;
            sayacButon->setText(
                QString("Tikla: %1").arg(sayac));
            etiket->setText(
                QString("Toplam tiklama: %1").arg(sayac));
        });

    // Kapat butonuna uygulamayi kapatma bağlantısi
    QObject::connect(kapatButon, &QPushButton::clicked,
                     &app, &QApplication::quit);

    pencere.show();
    return app.exec();
}
```

Lambda ile Sinyal/Slot

Lambda kullanırken yakalama listesine dikkat edin:

- `[=]` – Tüm değişkenleri değer ile yakala (kopyala)
- `[&]` – Tüm değişkenleri referans ile yakala
- `[sayac, buton]` – Sadece belirtilenleri yakala

Widget işaretçileri (`QPushButton*` vb.) değer ile yakalanmalıdır çünkü zaten birer adrestir.

Görev 3.1 – Temel Sinyal/Slot Bağlantıları (10 dk)

Başlangıç kodunu genişleterek:

a) Bir `"Sifirla"` butonu ekleyin. Tıklandığında sayacı 0'a döndürsün ve etiket ile buton metnini güncellesin.

b) Sayaç 10'a ulaştığında `sayacButon`'u devre dışı bırakın:

```
sayacButon->setEnabled(false);
```

c) Sıfırla butonu tıklandığında `sayacButon`'u tekrar etkinleştirin.

Görev 3.2 – Çoklu Sinyal Bağlantıları (10 dk)

Bir sinyali birden fazla slota bağlayın:

a) Yeni bir `"Bilgi"` butonu ekleyin.

b) Bu butonun `clicked` sinyalini 3 farklı lambda'ya bağlayın:

- Konsola `qDebug() « "Bilgi butonuna tiklandi";` yazdırsın.
- Etiketteki metni `"Bilgi butonu tiklandi"` olarak değiştirsin.
- Pencerenin başlığını `"OBYS - Bilgi Goruntulendi"` yapsın.

c) Butona tıklayarak 3 slot'un da çalıştığını doğrulayın.

Görev 3.3 – disconnect Kullanımı (10 dk)

Sinyal bağlantılarını dinamik olarak yönetin:

a) `connect()` çağrısının dönüş değerini saklayın:

```
auto bağlantı = QObject::connect(buton, &QPushButton::clicked,
    []() { qDebug() << "Bagli!"; });
```

b) Bir `"Baglantiyi Kes"` butonu ekleyin. Tıklandığında:

```
QObject::disconnect(bağlantı);
```

c) Bağlantı kesildikten sonra butonun tıklanmasının artık slot'u tetiklemediğini doğrulayın.

Dikkat: Bağlantı Yaşam Süresi

Bir nesne yok edildiğinde, o nesneyle ilgili tüm sinyal/slot bağlantıları otomatik olarak kesilir. Bu nedenle parent-child ilişkisini doğru kurduğunuzda bağlantı yönetimi büyük ölçüde otomatik olur. Ancak lambda ile bağlantılarda yakalanan değişkenlerin yaşam süresine dikkat edin!

4 Çalışma 4: Widget'larla Form Oluşturma (30 dk)

Ön Bilgi

Qt'da form tasarlamak için çeşitli giriş widget'ları kullanılır: `QLineEdit` (metin girişi), `QComboBox` (açılır liste), `QSpinBox` (sayı seçici), `QCheckBox` (onay kutusu). Bu widget'lar **düzen yöneticileri** (layout managers) ile düzenlenir: `QVBoxLayout` (dikey), `QHBoxLayout` (yatay), `QFormLayout` (etiket-alan çiftleri), `QGridLayout` (ızgara).

Aşağıdaki kodu `main.cpp` dosyasına yazın. Önce pencere ve form alanlarını oluşturuyoruz:

```
#include <QApplication>
#include <QWidget>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QFormLayout>
#include <QLineEdit>
#include <QComboBox>
#include <QSpinBox>
#include <QPushButton>
#include <QLabel>
#include <QDebug>

int main(int argc, char* argv[]) {
    QApplication app(argc, argv);

    QWidget pencere;
    pencere.setWindowTitle("OBYS - Ders Kayit Formu");
    pencere.resize(450, 350);

    // Ana duzen: dikey
    auto* anaDuzen = new QVBoxLayout(&pencere);

    // Baslik
    auto* baslik = new QLabel("Ders Kayit Formu");
    baslik->setStyleSheet(
        "font-size: 16px; font-weight: bold; color: #008080;");
    baslik->setAlignment(Qt::AlignCenter);
    anaDuzen->addWidget(baslik);
```

Form alanlarını `QFormLayout` ile ekliyoruz – etiket ve widget çiftlerini otomatik hizalar:

```
    // Form alanlari
    auto* formDuzen = new QFormLayout;

    auto* dersAdi = new QLineEdit;
    dersAdi->setPlaceholderText("Ders adini girin...");
    formDuzen->addRow("Ders Adi:", dersAdi);

    auto* bölüm = new QComboBox;
    bölüm->addItems({"Bilgisayar Bilimleri",
                     "Yazilim Muhendisligi",
                     "Elektrik-Elektronik",
                     "Makine Muhendisligi"});
    formDuzen->addRow("Bölüm:", bölüm);

    auto* kredi = new QSpinBox;
    kredi->setRange(1, 6);
```

```
    kredi->setValue(3);
    formDuzen->addRow("Kredi:", kredi);

    anaDuzen->addLayout(formDuzen);
```

Butonları yatay düzenle yerleştirip sinyal/slot bağlantılarını kuruyoruz:

```
    // Butonlar (yatay duzen)
    auto* butonDuzen = new QHBoxLayout;
    auto* kaydetButon = new QPushButton("Kaydet");
    auto* temizleButon = new QPushButton("Temizle");
    butonDuzen->addWidget(kaydetButon);
    butonDuzen->addWidget(temizleButon);
    anaDuzen->addLayout(butonDuzen);

    // Sonuc etiketi
    auto* sonuc = new QLabel;
    sonuc->setAlignment(Qt::AlignCenter);
    anaDuzen->addWidget(sonuc);

    // Sinyal/slot bağlantıları
    QObject::connect(kaydetButon, &QPushButton::clicked,
        [=]() {
            QString mesaj = QString("Kaydedildi: %1 (%2, %3 kredi)")
                .arg(dersAdi->text())
                .arg(bölüm->currentText())
                .arg(kredi->value());
            sonuc->setText(mesaj);
            qDebug() << mesaj;
        });

    QObject::connect(temizleButon, &QPushButton::clicked,
        [=]() {
            dersAdi->clear();
            bölüm->setCurrentIndex(0);
            kredi->setValue(3);
            sonuc->clear();
        });

    pencere.show();
    return app.exec();
}
```

Layout Yöneticileri

Layout'lar widget'ları otomatik konumlandırır ve pencere yeniden boyutlandırıldığında düzeni korur. `setGeometry()` ile manuel konumlandırma yerine layout kullanmak tercih edilir:

- `QVBoxLayout` – Widget'ları dikey sıralar
- `QHBoxLayout` – Widget'ları yatay sıralar
- `QFormLayout` – Etiket-alan çiftleri oluşturur
- `QGridLayout` – Satır/sütun ızgarası oluşturur

Görev 4.1 – Form Genişletme (15 dk)

Ders kayıt formuna aşağıdaki alanları ekleyin:

a) `QComboBox` ile `"Yariyil"` seçimi ekleyin (Güz, Bahar, Yaz).

b) `QSpinBox` ile `"Kontenjan"` alanı ekleyin (aralık: 10–200, varsayılan: 40).

c) `QCheckBox` ile `"Zorunlu Ders"` onay kutusu ekleyin:

```
#include <QCheckBox>
auto* zorunlu = new QCheckBox("Zorunlu Ders");
formDuzen->addRow("", zorunlu);
```

d) Kaydet butonunun slot'unu yeni alanları da içerecek şekilde güncelleyin.

Görev 4.2 – Doğrulama ve Geri Bildirim (15 dk)

Form doğrulaması ve kullanıcı geri bildirimi ekleyin:

a) Kaydet butonuna tıklandığında ders adının boş olup olmadığını kontrol edin. Boşsa sonuç etiketinde kırmızı renkte uyarı gösterin:

```
if (dersAdi->text().isEmpty()) {
    sonuc->setStyleSheet("color: red; font-weight: bold;");
    sonuc->setText("Hata: Ders adi bos olamaz!");
    return;
}
```

b) Geçerli kayıt durumunda sonuç etiketini yeşil renkle gösterin:

```
sonuc->setStyleSheet("color: green; font-weight: bold;");
```

c) `dersAdi` alanının `textChanged` sinyalini kullanarak, her karakter girişinde ders adının uzunluğunu bir etikette gösterin (ör: `"12 / 50 karakter"`).

5 Çalışma 5: OBYS Kayıt Formu (20 dk)

Ön Bilgi

Bu çalışmada önceki bölümlerde öğrendiklerinizi birleştirerek eksiksiz bir OBYS öğrenci kayıt formu oluşturacaksınız. Form doğrulama, sinyal/slot bağlantıları ve düzen yönetimini bir arada kullanacaksınız.

Görev 5.1 – OBYS Öğrenci Kayıt Formu (20 dk)

Aşağıdaki alanları içeren tam bir öğrenci kayıt formu oluşturun:

a) Pencere başlığı: `"OBYS - Öğrenci Kayit Formu"`

b) Form alanları:

- `QLineEdit` – Öğrenci No (sadece rakam, `setMaxLength(9)` ile sınırlı)
- `QLineEdit` – Ad Soyad
- `QComboBox` – Bölüm (en az 4 bölüm)
- `QSpinBox` – Sınıf (1–4)
- `QComboBox` – Yarıyıl (Güz / Bahar)

c) Butonlar: `"Kaydet"` ve `"Temizle"`

d) Doğrulama kuralları:

- Öğrenci No boş olamaz ve en az 6 karakter olmalı
- Ad Soyad boş olamaz
- Kayıt başarılıysa yeşil, hatalıysa kırmızı mesaj

e) Kaydet butonuna tıklandığında tüm bilgileri `qDebug()` ile konsola yazdırın.

f) Düzen için `QFormLayout` ve `QVBoxLayout` kullanın (manuel `setGeometry()` kullanmayın).

```
// Ipucu: Öğrenci numarasi doğrulama
auto* ogrNo = new QLineEdit;
ogrNo->setMaxLength(9);
ogrNo->setPlaceholderText("Ornek: 123456789");
```

Dikkat: QString ile int Dönüşümü

`QLineEdit`'ten gelen değer `QString` türündedir. Sayıya dönüştürmek için `toInt()` kullanabilirsiniz:
`int no = ogrNo->text().toInt();`
Ancak bu dönüşüm başarısız olabilir. Güvenli kontrol için:
`bool ok; int no = ogrNo->text().toInt(&ok);`

6 Çalışma 6: QString ve Qt Kapsayıcıları (20 dk)

Ön Bilgi

Qt'nin temel veri tipleri STL karşılıklarına benzer ancak implicit sharing (copy-on-write) sayesinde değer olarak geçirilmeleri ucuzdur. `QString` Qt'nin UTF-16 metin sınıfıdır; `QStringList`, `QList`, `QMap` ve `QSet` ise en çok kullanılan Qt kapsayıcılarıdır.

Aşağıdaki başlangıç kodunu `main.cpp` dosyasına yazın:

```
#include <QCoreApplication>
#include <QString>
#include <QStringList>
#include <QList>
#include <QMap>
#include <QDebug>

int main(int argc, char* argv[]) {
    QCoreApplication app(argc, argv);

    // QString temel islemler
    QString isim = "Ali Yilmaz";
    qDebug() << "Isim:" << isim;
    qDebug() << "Uzunluk:" << isim.size();
    qDebug() << "Buyuk harf:" << isim.toUpper();

    // arg() ile formatlama
    QString bilgi = QString("Öğrenci: %1, No: %2")
                        .arg(isim).arg(1001);
    qDebug() << bilgi;

    // split ve QStringList
    QString csv = "Algoritma,Veritabani,Yazilim";
    QStringList dersler = csv.split(",");
    qDebug() << "Dersler:" << dersler;
    qDebug() << "Join:" << dersler.join(" | ");

    // QMap ile öğrenci sozlugu
    QMap<int, QString> öğrenciler;
    öğrenciler[1001] = "Ali Yilmaz";
    öğrenciler[1002] = "Ayse Kaya";
    qDebug() << "1001:" << öğrenciler.value(1001);
    qDebug() << "Anahtarlar:" << öğrenciler.keys();

    return 0;
}
```

Bu çalışma GUI gerektirmez

Bu çalışmada `QCoreApplication` kullanılır (GUI yok). CMakeLists.txt'te `Qt6::Widgets` yerine `Qt6::Core` yeterlidir:

```
find_package(Qt6 REQUIRED COMPONENTS Core)
target_link_libraries(app PRIVATE Qt6::Core)
```

Görev 6.1 – QString İşlemleri (10 dk)

`QString`'in metin işleme yeteneklerini keşfedin:

a) Bir `isimDogrula()` fonksiyonu yazın: parametre olarak `const QString&` alsın, ismin boş olmadığını ve en az 2 kelimeden (ad+soyad) oluştuğunu kontrol etsin. `trimmed()`, `split()`, `isEmpty()` kullanın.

b) `arg()` ile OBYS bilgi kartı oluşturun:

```
QString kart = QString("Öğrenci: %1\nNo: %2\nNot: %3")
    .arg(isim).arg(numara).arg(ortalama, 0, 'f', 2);
```

c) `std::string` ile dönüşüm yapın: `toStdString()` ve `fromStdString()` kullanın.

Görev 6.2 – Qt Kapsayıcıları ile OBYS Sözlüğü (10 dk)

Qt kapsayıcılarını OBYS bağlamında kullanın:

a) `QMap<int, QString>` ile 5 öğrencilik bir sözlük oluşturun. `insert()` ve `operator[]` ile ekleme yapın.

b) Var olmayan bir numarayı `value(numara, "Bilinmiyor")` ile arayın.

c) `QList<int>` ile not listesi oluşturun. STL algoritması `std::accumulate()` kullanarak ortalama hesaplayın.

d) `QSet<QString>` ile ders kümesi oluşturup iki kümenin kesişimini (`intersect()`) bulun.

Dikkat: QMap operator[

vs value()] `QMap`'te `operator[]` ile var olmayan bir anahtara erişirseniz, o anahtar varsayılan değerle **eklenir**. Sadece okuma yapacaksanız `value()` kullanın – bu, map'e yeni giriş eklemez.

Teslim Bilgileri

Teslim edilecekler:

1. `çalışma1/` – İlk Qt penceresi (Görev 1.1–1.2)
2. `çalışma2/` – Parent-Child ve QObject (Görev 2.1–2.2)
3. `çalışma3/` – Sinyal/Slot bağlantıları (Görev 3.1–3.3)
4. `çalışma4/` – Widget'larla form oluşturma (Görev 4.1–4.2)
5. `çalışma5/` – OBYS kayıt formu (Görev 5.1)
6. `çalışma6/` – QString ve Qt kapsayıcıları (Görev 6.1–6.2)

Her çalışma klasörü bir `CMakeLists.txt` ve `main.cpp` dosyası içermelidir.
Puanlama: Çalışma 1: 10p Çalışma 2: 15p Çalışma 3: 25p Çalışma 4: 20p Çalışma 5: 15p Çalışma 6: 15p